

Introduction to Generative AI in CV

Lecture 1 — From Autoencoders to Hierarchical VAEs

Levon Khachatryan

Senior Machine Learning Researcher at **Picsart**
Computer Vision Lecturer at **Yerevan State University**

levon-kh.github.io



Picsart

Overview

- 1 Introduction
- 2 Challenges in Modeling Distributions
- 3 Variational Perspective: From VAEs to DDPMs

Generative AI in Action

Examples from Picasso's paintings



illustrative examples from the training collection

learn patterns
and generate



AI-generated samples



newly generated samples

The model learns recurring **shapes, colors, and compositions** from the paintings, then combines them in new ways to create **new images**.

From Text to Images



Text prompt

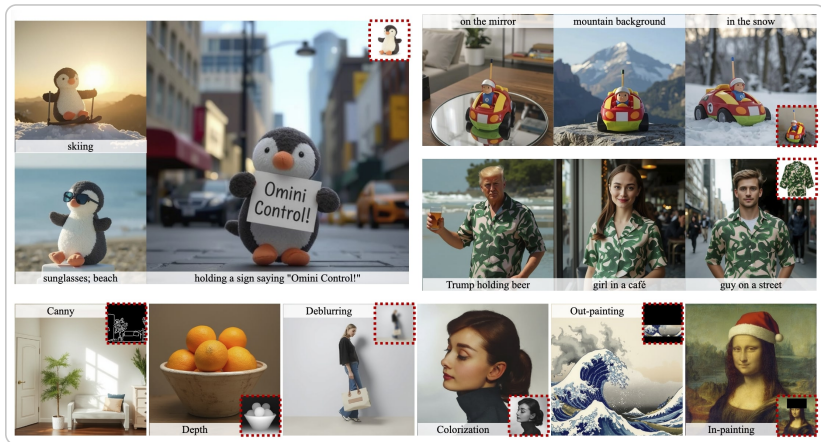


generated image

Example prompt: "An astronaut surrounded by colorful cosmic clouds."

Image source: [Midjourney](#)

More Control over Generation



Text prompt → what to generate

Visual guidance → where and how it should appear

Image source: Tan et al., *OmniControl* (2024).

What Do These Examples Have in Common?

The previous examples differ in the amount of guidance provided. However, mathematically, they share the same basic idea:

Generating a new example means sampling one possible outcome from a probability distribution.

Without guidance: $\tilde{\mathbf{x}} \sim p(\mathbf{x}),$ *unconditional generation,*

With guidance c : $\tilde{\mathbf{x}} \sim p(\mathbf{x} \mid c),$ *conditional generation.*

Here, c can be a text prompt, a reference image, a mask, or another form of visual guidance.

To understand generation, let us first see how we generate samples when the probability distribution is already known.

Known Distributions Can Generate New Samples

Example: Bernoulli distribution.

Suppose a coin has $\Pr(X = \text{Head}) = p$, $\Pr(X = \text{Tail}) = 1 - p$.

To generate one outcome, draw $U \sim \text{Uniform}(0, 1)$, and set

$$X = \begin{cases} \text{H}, & U \leq p, \\ \text{T}, & U > p. \end{cases}$$

For example, if $p = 0.6$,

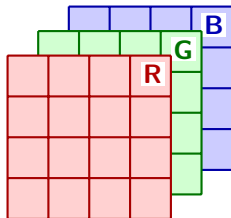
U	0.23	0.81	0.54	0.12	0.70
X	H	T	H	H	T

Knowing the distribution allows us to generate samples with the desired statistical behavior.

For Images, the Distribution Is Unknown

An RGB image is **not a single outcome such as** Head **or** Tail. It is a high-dimensional random vector containing $3HW$ values:

$$\mathbf{x} \in \{0, \dots, 255\}^{H \times W \times 3}$$



One image is one joint assignment of all $3HW$ channel values.

With the coin, we explicitly specified the probability of every outcome. For real images, we observe only a finite dataset:

$$\mathcal{D} = \{\mathbf{x}^{(i)}\}_{i=1}^n, \quad \mathbf{x}^{(i)} \stackrel{\text{i.i.d.}}{\sim} p_{\text{data}}(\mathbf{x}).$$

The distribution $p_{\text{data}}(\mathbf{x})$ that generated these observations is unknown:

- we do not know its probability density;
- we do not have its sampling procedure.

Can we learn a distribution—and its sampling procedure—from observed examples alone?

Deep Generative Modeling

Definition

Given observations

$$\mathcal{D} = \{\mathbf{x}^{(i)}\}_{i=1}^n, \quad \mathbf{x}^{(i)} \stackrel{\text{i.i.d.}}{\sim} p_{\text{data}}(\mathbf{x}),$$

deep generative modeling uses neural networks to learn a $p_{\phi}(\mathbf{x})$ that approximates the unknown data distribution $p_{\text{data}}(\mathbf{x})$.

The objective is therefore

$$\boxed{p_{\phi}(\mathbf{x}) \approx p_{\text{data}}(\mathbf{x})} \implies \tilde{\mathbf{x}} \sim p_{\phi}(\mathbf{x}).$$

A successful deep generative model should support:

- 1 **Realistic and diverse generation:** samples from p_{ϕ} should reproduce the important characteristics and variability of real data.
- 2 **Controllable generation:** the generation process should allow meaningful control over properties such as content, structure, style, or other desired attributes.

We Need to Measure Distributional Mismatch

Our goal is $p_\phi(\mathbf{x}) \approx p_{\text{data}}(\mathbf{x})$. To learn p_ϕ , we need a measure of their discrepancy.

A commonly used discrepancy is the **forward KL divergence**:

$$\mathcal{D}_{\text{KL}}(p_{\text{data}} \| p_\phi) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} \left[\log \frac{p_{\text{data}}(\mathbf{x})}{p_\phi(\mathbf{x})} \right]$$

Why is this a meaningful choice?

[▶ See the Proofs](#)

- 1 It is always non-negative:

$$\mathcal{D}_{\text{KL}}(p_{\text{data}} \| p_\phi) \geq 0$$

- 2 It equals zero only when the model matches the data distribution:

$$\mathcal{D}_{\text{KL}}(p_{\text{data}} \| p_\phi) = 0 \iff p_\phi = p_{\text{data}}$$

- 3 Assigning much less density than p_{data} to real data incurs a large penalty:

$$p_\phi(\mathbf{x}) \ll p_{\text{data}}(\mathbf{x}) \implies \log \frac{p_{\text{data}}(\mathbf{x})}{p_\phi(\mathbf{x})} \text{ large}$$

Forward KL Minimization Leads to MLE

Expanding the forward KL gives

$$\mathcal{D}_{\text{KL}}(p_{\text{data}} \| p_{\phi}) = \underbrace{\mathbb{E}_{p_{\text{data}}} [\log p_{\text{data}}(\mathbf{x})]}_{\text{constant in } \phi} - \underbrace{\mathbb{E}_{p_{\text{data}}} [\log p_{\phi}(\mathbf{x})]}_{\text{expected log-likelihood}}.$$

Since the first term does not depend on ϕ ,

$$\boxed{\min_{\phi} \mathcal{D}_{\text{KL}}(p_{\text{data}} \| p_{\phi}) \iff \max_{\phi} \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} [\log p_{\phi}(\mathbf{x})]}$$

For i.i.d. observations $\mathcal{D} = \{\mathbf{x}^{(i)}\}_{i=1}^n$, we estimate the expectation using their sample average. This gives the **Maximum Likelihood Estimation** objective:

$$\boxed{\phi_{\text{MLE}}^* \in \arg \max_{\phi} \frac{1}{n} \sum_{i=1}^n \log p_{\phi}(\mathbf{x}^{(i)})}$$

MLE learns p_{ϕ} by assigning high likelihood to the observed data.

Overview

- 1 Introduction
- 2 Challenges in Modeling Distributions
- 3 Variational Perspective: From VAEs to DDPMs

Recall: PDF and PMF

A probability density function $p(x)$ describes the distribution of a Continuous Random Variable X , i.e.

$$\mathbb{P}(a \leq X \leq b) = \int_a^b p(x) \, dx$$

and we have:

- ① **Non-negativity:** $p(x) \geq 0$ for all x in the domain.
- ② **Normalization:** integral over the entire domain is equal 1, i.e. $\int p(x) dx = 1$

When X is discrete then we call $p(x)$ probability mass function mapping each possible outcome to its probability of occurrence

$$\mathbb{P}(X = x) = p(x)$$

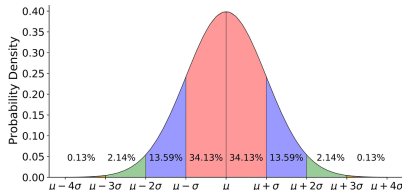
and we have:

- ① **Non-negativity:** $p(x) \geq 0$ for all x in the domain.
- ② **Normalization:** Sum over the entire domain is equal 1, i.e. $\sum_{x \in \mathcal{X}} p(x) = 1$

Examples of a PDF and a PMF

1. Gaussian PDF

Let X be a continuous random variable: $X \sim \mathcal{N}(\mu, \sigma^2)$.



$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right), \quad x \in \mathbb{R}.$$

Probabilities are areas under the density:

$$\mathbb{P}(a \leq X \leq b) = \int_a^b p(x) \, dx.$$

2. Fair-coin PMF

Let $X \in \{H, T\}$ denote the outcome of one coin toss:

$$p(x) = \begin{cases} \frac{1}{2}, & x = H, \\ \frac{1}{2}, & x = T, \\ 0, & \text{otherwise.} \end{cases}$$

Constructing a Valid Neural Probability Density

A natural idea is to approximate $p_{\text{data}}(\mathbf{x})$ using a neural network $f_{\phi}(\mathbf{x}) \approx p_{\text{data}}(\mathbf{x})$. However, a general NN represents an unconstrained real-valued function. Therefore, f_{ϕ} is not automatically a valid probability density: it may be negative, and its integral need not equal one.

Step 1: Ensure non-negativity.

Apply a non-negative function to the raw network output. Possible choices include

$$|f_{\phi}(\mathbf{x})|, \quad f_{\phi}^2(\mathbf{x}), \quad \exp(f_{\phi}(\mathbf{x})).$$

A standard choice is the exponential, which defines the *unnormalized density*

$$\tilde{p}_{\phi}(\mathbf{x}) := \exp(f_{\phi}(\mathbf{x})) > 0.$$

Step 2: Enforce normalization.

The function $\tilde{p}_{\phi}$ is positive but may not integrate to one. We therefore divide it by its integral over the data space:

$$p_{\phi}(\mathbf{x}) = \frac{\tilde{p}_{\phi}(\mathbf{x})}{\int \tilde{p}_{\phi}(\mathbf{x}') \, d\mathbf{x}'} = \frac{\exp(f_{\phi}(\mathbf{x}))}{\int \exp(f_{\phi}(\mathbf{x}')) \, d\mathbf{x}'}$$

The denominator is the *normalizing constant* denoted by $Z(\phi)$.

The Normalizing Constant Is the Computational Bottleneck

Maximum Likelihood Estimation requires evaluating the model log-likelihood. From the previous slide,

$$p_{\phi}(\mathbf{x}) = \frac{\exp(f_{\phi}(\mathbf{x}))}{Z(\phi)},$$

and therefore

$$\log p_{\phi}(\mathbf{x}) = \underbrace{f_{\phi}(\mathbf{x})}_{\text{easy to evaluate with the NN}} - \underbrace{\log Z(\phi)}_{\text{difficult to compute}}.$$

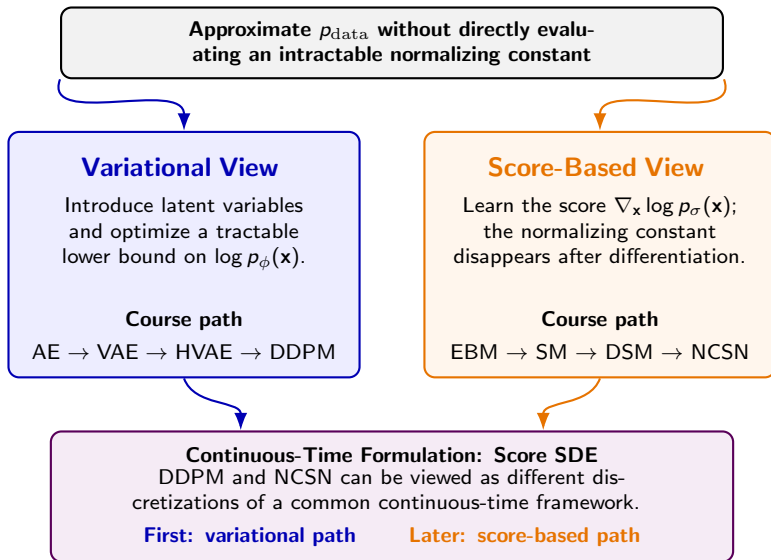
The normalizing constant must account for every possible observation $\mathbf{x}'$ in the data space. For example 256×256 RGB image lies in

$$\mathbf{x} \in \{0, 1, \dots, 255\}^{256 \cdot 256 \cdot 3}$$

Thus, evaluating $Z(\phi)$ requires computing a 196,608-dimensional integral, which is infeasible in such a high-dimensional space.

Direct neural density modeling is mathematically valid, but exact likelihood evaluation is generally intractable for high-dimensional data.

Two Perspectives Used in This Course



Overview

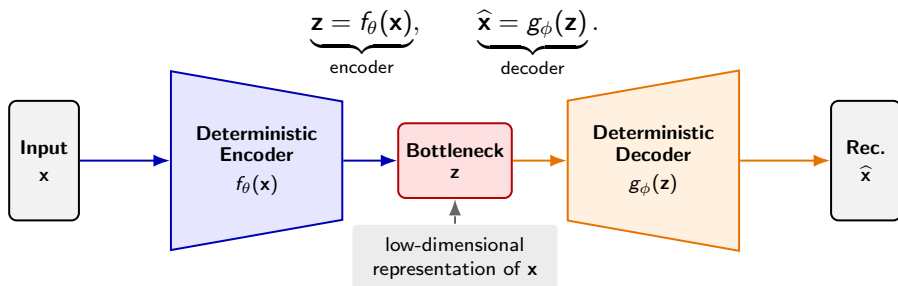
- 1 Introduction
- 2 Challenges in Modeling Distributions
- 3 Variational Perspective: From VAEs to DDPMs
 - Autoencoders
 - Variational Autoencoders (VAEs)
 - Hierarchical Variational Autoencoders (HVAE)

Outline

- 1 Introduction
- 2 Challenges in Modeling Distributions
- 3 Variational Perspective: From VAEs to DDPMs
 - Autoencoders
 - Variational Autoencoders (VAEs)
 - Hierarchical Variational Autoencoders (HVAE)

An Autoencoder

An autoencoder learns to reconstruct an observation through a **compact latent representation**. It consists of two deterministic neural networks:



Training objective. The encoder and decoder are trained jointly to reconstruct the observations:

$$(\theta^*, \phi^*) \in \arg \min_{\theta, \phi} \mathcal{L}_{\text{AE}}(\theta, \phi); \quad \mathcal{L}_{\text{AE}}(\theta, \phi) = \frac{1}{n} \sum_{i=1}^n \left\| \mathbf{x}^{(i)} - g_{\phi} \left(f_{\theta}(\mathbf{x}^{(i)}) \right) \right\|_2^2$$

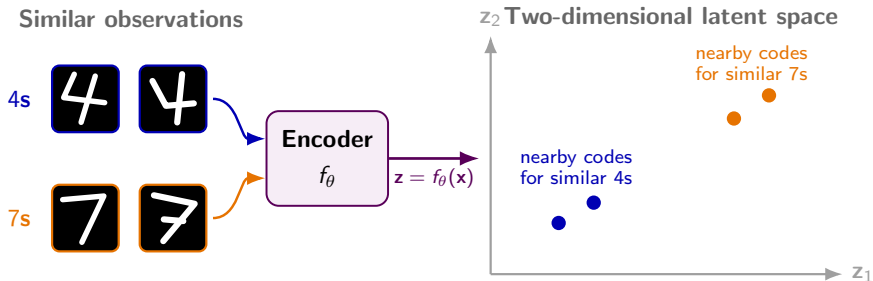
The latent code must preserve the information needed to reconstruct the input.

What Does the Bottleneck Represent?

Because the latent code is lower-dimensional than the input,

$$\dim(\mathbf{z}) \ll \dim(\mathbf{x}),$$

the encoder cannot preserve every input value. It must learn a compact code that retains the information needed for reconstruction.

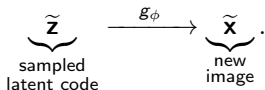


The latent space summarizes reconstruction-relevant similarities: similar observations often receive nearby latent codes.

Could We Generate from the Bottleneck?

Returning to the Picasso example, imagine that a latent code represents one combination of recurring **shapes, colors, and compositions**.

If we could sample a meaningful latent code, generation would seem simple:



The decoder would transform the selected combination of visual properties into an image.



Hypothetical decoded samples

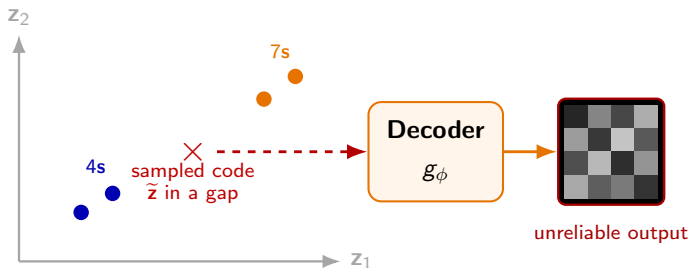
Missing piece: Which latent codes are meaningful, and from what distribution should we sample $\tilde{\mathbf{z}}$?

Why Is a Standard Autoencoder Not Generative?

During training, each observation is mapped to one latent code, and the decoder learns to reconstruct the observation from that code:

$$\mathbf{x}^{(i)} \xrightarrow{f_\theta} \mathbf{z}^{(i)} \xrightarrow{g_\phi} \hat{\mathbf{x}}^{(i)}.$$

For generation, we must select a new code $\tilde{\mathbf{z}}$. A standard AE provides no rule for determining which other points in the latent space represent meaningful images.

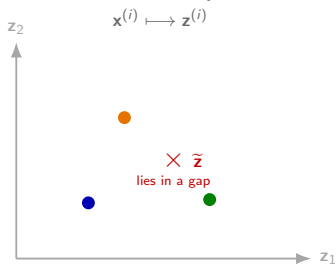


A randomly chosen code may lie far from the training codes, so its decoded output can be unreliable.

The Key Change: From a Point to a Distribution

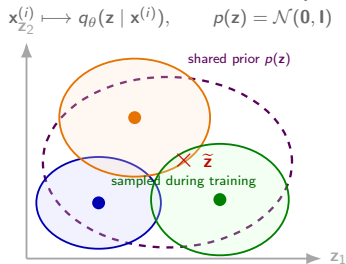
A VAE replaces each deterministic code with a **distribution over nearby codes** and organizes these distributions around a **shared prior**.

Standard AE: one code per observation



Only isolated codes are used during training.

VAE: distribution and shared prior



Ellipses: encoding distributions; dashed: high-density prior region.

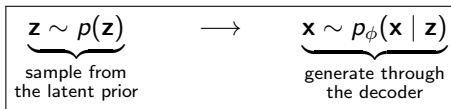
A VAE trains the decoder on latent neighborhoods and regularizes those neighborhoods toward a shared prior used for generation.

Outline

- 1 Introduction
- 2 Challenges in Modeling Distributions
- 3 Variational Perspective: From VAEs to DDPMs**
 - Autoencoders
 - Variational Autoencoders (VAEs)**
 - Hierarchical Variational Autoencoders (HVAE)

A VAE Defines a Probabilistic Generative Model

A VAE describes generation as a two-step stochastic process:



The prior is commonly chosen as $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$, while the decoder network parameterizes the conditional distribution $p_{\phi}(\mathbf{x} | \mathbf{z})$.

Together they define the joint distribution $p_{\phi}(\mathbf{x}, \mathbf{z}) = p_{\phi}(\mathbf{x} | \mathbf{z}) p(\mathbf{z})$.

A particular observation may have many possible latent explanations. Because $\mathbf{z}$ is unobserved, the model likelihood of $\mathbf{x}$ accounts for every possible latent value:

$$p_{\phi}(\mathbf{x}) = \int \underbrace{p_{\phi}(\mathbf{x} | \mathbf{z})}_{\text{how well } \mathbf{z} \text{ explains } \mathbf{x}} \underbrace{p(\mathbf{z})}_{\text{how plausible } \mathbf{z} \text{ is}} d\mathbf{z}.$$

The prior tells us where to sample latent codes; the decoder tells us what observations those codes can generate.

Learning Requires Inference in the Reverse Direction

The prior and decoder describe generation from $\mathbf{z}$ to $\mathbf{x}$. During training, however, we observe only $\mathbf{x}$; the latent value $\mathbf{z}$ that may have generated it is unknown.

To connect the observation to the generative model, we ask:

Which latent values could plausibly have generated this $\mathbf{x}$?

Bayes' rule defines the corresponding posterior distribution:

$$p_{\phi}(\mathbf{z} | \mathbf{x}) = \frac{p_{\phi}(\mathbf{x} | \mathbf{z}) p(\mathbf{z})}{p_{\phi}(\mathbf{x})}$$

For a fixed $\mathbf{x}$, the numerator assigns each latent value an unnormalized weight:

$$\underbrace{p_{\phi}(\mathbf{x} | \mathbf{z})}_{\text{how well } \mathbf{z} \text{ explains } \mathbf{x}} \quad \underbrace{p(\mathbf{z})}_{\text{how likely } \mathbf{z} \text{ is under the prior}}$$

The denominator normalizes scores by integrating over all possible latent values:

$$p_{\phi}(\mathbf{x}) = \int p_{\phi}(\mathbf{x} | \mathbf{z}') p(\mathbf{z}') d\mathbf{z}'.$$

This integral generally has no closed-form solution, meaning the exact posterior cannot be computed.

The Variational Encoder Approximates the Posterior

Because the exact posterior is intractable, a VAE introduces a tractable approximation:

$$q_{\theta}(\mathbf{z} \mid \mathbf{x}) \approx p_{\phi}(\mathbf{z} \mid \mathbf{x}).$$

As stated before, instead of assigning a single latent code to $\mathbf{x}$, the VAE encoder assigns a distribution over possible latent codes.

Given $\mathbf{x}$, the encoder neural network outputs two vectors:

$$\mathbf{x} \xrightarrow{\text{encoder NN}} (\mu_{\theta}(\mathbf{x}), \sigma_{\theta}(\mathbf{x})).$$

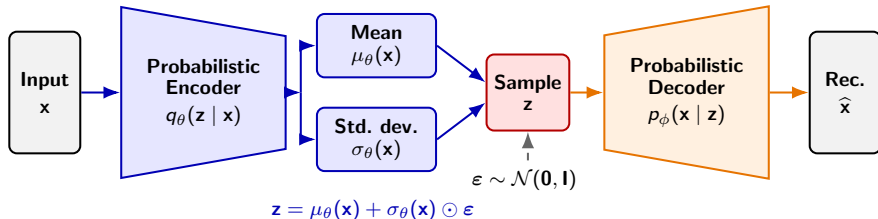
These vectors define a Gaussian approximate posterior:

$$q_{\theta}(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}(\mathbf{z}; \mu_{\theta}(\mathbf{x}), \text{diag}(\sigma_{\theta}^2(\mathbf{x}))).$$

- $\mu_{\theta}(\mathbf{x})$ specifies the center of the plausible latent region.
- $\sigma_{\theta}(\mathbf{x})$ specifies the uncertainty in each latent dimension.

The diagonal covariance assumes conditional **independence between latent dimensions**, making sampling and KL-divergence computation efficient.

Putting the Two Parts Together: The VAE



- **Inference model:** the encoder outputs $\mu_{\theta}(\mathbf{x})$ and $\sigma_{\theta}(\mathbf{x})$, which parameterize $q_{\theta}(\mathbf{z} | \mathbf{x})$.
- **Latent sampling:** to sample from $q_{\theta}(\mathbf{z} | \mathbf{x})$, draw $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and compute

$$\mathbf{z} = \mu_{\theta}(\mathbf{x}) + \sigma_{\theta}(\mathbf{x}) \odot \epsilon.$$

- **Generative model:** the decoder specifies $p_{\phi}(\mathbf{x} | \mathbf{z})$. For generation, sample $\mathbf{z} \sim p(\mathbf{z})$ and then $\mathbf{x} \sim p_{\phi}(\mathbf{x} | \mathbf{z})$.

How can we jointly train the inference model q_{θ} and the generative model p_{ϕ} ?

Deriving the Evidence Lower Bound

Recall the marginal log-likelihood:

$$\log p_{\phi}(\mathbf{x}) = \log \int p_{\phi}(\mathbf{x}, \mathbf{z}) \, d\mathbf{z}.$$

Insert the encoder distribution by multiplying and dividing by $q_{\theta}(\mathbf{z} \mid \mathbf{x})$:

$$\log p_{\phi}(\mathbf{x}) = \log \int q_{\theta}(\mathbf{z} \mid \mathbf{x}) \frac{p_{\phi}(\mathbf{x}, \mathbf{z})}{q_{\theta}(\mathbf{z} \mid \mathbf{x})} \, d\mathbf{z} = \log \mathbb{E}_{\mathbf{z} \sim q_{\theta}(\mathbf{z} \mid \mathbf{x})} \left[\frac{p_{\phi}(\mathbf{x}, \mathbf{z})}{q_{\theta}(\mathbf{z} \mid \mathbf{x})} \right]$$

Since $\log(\cdot)$ is concave, Jensen's inequality gives

$$\log \mathbb{E}[Y] \geq \mathbb{E}[\log Y]$$

► Recall Jensen's inequality

Applying it yields

$$\log p_{\phi}(\mathbf{x}) \geq \underbrace{\mathbb{E}_{\mathbf{z} \sim q_{\theta}(\mathbf{z} \mid \mathbf{x})} \left[\log \frac{p_{\phi}(\mathbf{x}, \mathbf{z})}{q_{\theta}(\mathbf{z} \mid \mathbf{x})} \right]}_{\mathcal{L}_{\text{ELBO}}(\theta, \phi; \mathbf{x})}$$

The ELBO can be estimated using samples from $q_{\theta}(\mathbf{z} \mid \mathbf{x})$ without integrating over all latent values.

The ELBO Balances Two Objectives

Using the generative factorization

$$p_{\phi}(\mathbf{x}, \mathbf{z}) = p_{\phi}(\mathbf{x} \mid \mathbf{z}) p(\mathbf{z}),$$

the ELBO becomes

$$\begin{aligned} \mathcal{L}_{\text{ELBO}} &= \mathbb{E}_{q_{\theta}(\mathbf{z} \mid \mathbf{x})} \left[\log p_{\phi}(\mathbf{x} \mid \mathbf{z}) + \log \frac{p(\mathbf{z})}{q_{\theta}(\mathbf{z} \mid \mathbf{x})} \right] \\ &= \underbrace{\mathbb{E}_{q_{\theta}(\mathbf{z} \mid \mathbf{x})} [\log p_{\phi}(\mathbf{x} \mid \mathbf{z})]}_{\text{expected reconstruction log-likelihood}} - \underbrace{\mathcal{D}_{\text{KL}}(q_{\theta}(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z}))}_{\text{latent regularization}}. \end{aligned}$$

- **Reconstruction:** latent samples inferred from $\mathbf{x}$ should allow the decoder to assign high probability to that observation.
- **Latent regularization:** the inferred distribution should remain close to the prior, so that latent values drawn from $p(\mathbf{z})$ can be meaningfully decoded.

Reconstruction preserves information about $\mathbf{x}$; regularization makes sampling from the prior meaningful.

The VAE Training Loss In Practice

The encoder and decoder are trained jointly by **minimizing the negative ELBO**:

$$\mathcal{L}_{\text{VAE}}(\theta, \phi; \mathbf{x}) = \underbrace{-\mathbb{E}_{\mathbf{z} \sim q_{\theta}(\mathbf{z} | \mathbf{x})} [\log p_{\phi}(\mathbf{x} | \mathbf{z})]}_{\text{reconstruction loss}} + \underbrace{\mathcal{D}_{\text{KL}}(q_{\theta}(\mathbf{z} | \mathbf{x}) \| p(\mathbf{z}))}_{\text{KL loss}}$$

In practice, this loss is computed as follows:

Reconstruction loss. Draw one latent code $\mathbf{z} \sim q_{\theta}(\mathbf{z} | \mathbf{x})$, and estimate the reconstruction loss as

$$\hat{\mathcal{L}}_{\text{rec}} = -\log p_{\phi}(\mathbf{x} | \mathbf{z}).$$

KL loss. Let d denote the latent dimensionality. The KL term has a closed form:

$$\mathcal{L}_{\text{KL}} = \frac{1}{2} \sum_{j=1}^d \left[\underbrace{\mu_{\theta,j}^2(\mathbf{x})}_{\substack{\text{mean penalty:} \\ \mu_{\theta,j}(\mathbf{x}) \rightarrow 0}} + \underbrace{\sigma_{\theta,j}^2(\mathbf{x}) - \log \sigma_{\theta,j}^2(\mathbf{x}) - 1}_{\substack{\text{variance penalty:} \\ \sigma_{\theta,j}^2(\mathbf{x}) \rightarrow 1}} \right]$$

Results Generated by VAE



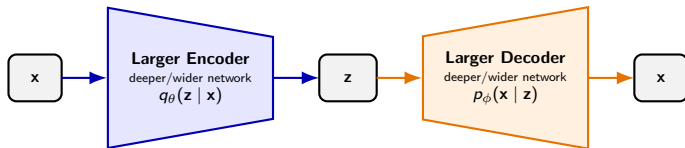
Figure: Results of VAE trained on CelebFaces dataset

VAEs provide a principled generative framework, but their limited modeling capacity can restrict generation quality.

How Can We Improve VAE Quality?

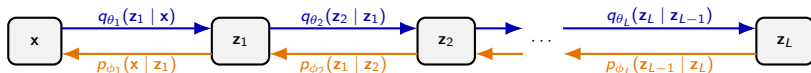
A natural question is: **how should we scale and improve the VAE?**

Option 1 — Vertical scaling: increase network capacity



- The encoder and decoder become deeper or wider, providing greater NN capacity.
- The probabilistic structure remains unchanged, with only **one stochastic latent variable** z .

Option 2 — Horizontal scaling: deepen the latent hierarchy



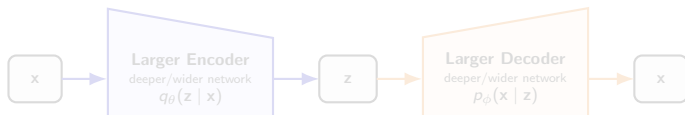
- The probabilistic structure is deeper via **multiple stochastic latent variables** $z_1, \dots, z_L$.
- Each transition is typically implemented by a stage-specific network block:

$$\theta = \{\theta_1, \dots, \theta_L\}, \quad \phi = \{\phi_1, \dots, \phi_L\}.$$

How Can We Improve VAE Quality?

A natural question is: **how should we scale and improve the VAE?**

Option 1 — Vertical scaling: increase network capacity



Audience question

What is the key advantage of introducing a hierarchy of stochastic latent variables instead of only enlarging the encoder and decoder?

- The probabilistic structure is deeper via **multiple stochastic latent variables** $z_1, \dots, z_L$.
- Each transition is typically implemented by a stage-specific network block:

$$\theta = \{\theta_1, \dots, \theta_L\}, \quad \phi = \{\phi_1, \dots, \phi_L\}.$$

Outline

- 1 Introduction
- 2 Challenges in Modeling Distributions
- 3 Variational Perspective: From VAEs to DDPMs**
 - Autoencoders
 - Variational Autoencoders (VAEs)
 - **Hierarchical Variational Autoencoders (HVAE)**

What Is the Main Advantage of an HVAE?

A flat VAE asks one latent variable to represent all sources of variation. An HVAE distributes stochastic choices across several conditional stages.

Conceptually, the levels can specialize as



Each lower level adds finer information based on the broader structure provided by the level above. For example,

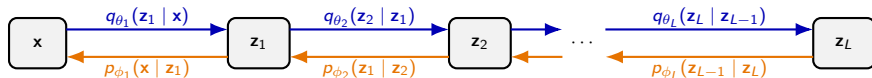
$$\underbrace{p(\text{global structure})}_{\text{coarse choice}} \underbrace{p(\text{shapes} \mid \text{global structure})}_{\text{intermediate choice}} \underbrace{p(\text{details} \mid \text{shapes})}_{\text{fine choice}}.$$

- Higher levels can capture broad, abstract variation.
- Lower levels can add local structure and fine details.
- Uncertainty can be represented at several levels instead of all at once.

This multiscale specialization is an inductive bias, not a mathematical guarantee.

From a Single Latent Variable to a Hierarchy

A Hierarchical VAE uses a sequence of stochastic latent variables $\mathbf{z}_1, \dots, \mathbf{z}_L$.



First-order Markov assumption. In each direction, a latent variable depends only on its immediate neighbor. The chain rule therefore gives

$$q_{\theta}(\mathbf{z}_{1:L} | \mathbf{x}) = q_{\theta}(\mathbf{z}_1 | \mathbf{x}) \prod_{i=2}^L q_{\theta}(\mathbf{z}_i | \mathbf{z}_{i-1}),$$
$$p_{\phi}(\mathbf{x}, \mathbf{z}_{1:L}) = p(\mathbf{z}_L) \prod_{i=2}^L p_{\phi}(\mathbf{z}_{i-1} | \mathbf{z}_i) p_{\phi}(\mathbf{x} | \mathbf{z}_1).$$

- **Inference model q_{θ} —left to right:** provides tractable latent distributions for an observed $\mathbf{x}$ and is used during training.
- **Generative model p_{ϕ} —right to left:** defines the distribution used for generation, beginning with $\mathbf{z}_L \sim p(\mathbf{z}_L)$ and ending with $\mathbf{x}$.

Notation. The diagram uses θ_i and ϕ_i for the parameters of individual stages. In the equations, θ and ϕ denote the corresponding collections of all stage parameters.

The Same Variational Principle Applies to an HVAE

An HVAE replaces the single latent variable $\mathbf{z}$ with many $\mathbf{z}_{1:L} = (\mathbf{z}_1, \dots, \mathbf{z}_L)$. Since these variables are unobserved, the marginal likelihood is obtained by integrating over the complete hierarchy:

$$p_{\phi}(\mathbf{x}) = \int \cdots \int p_{\phi}(\mathbf{x}, \mathbf{z}_{1:L}) \, d\mathbf{z}_1 \cdots d\mathbf{z}_L.$$

Applying the same variational argument as for a standard VAE, using the hierarchical encoder $q_{\theta}(\mathbf{z}_{1:L} \mid \mathbf{x})$, gives

$$\log p_{\phi}(\mathbf{x}) \geq \underbrace{\mathbb{E}_{\mathbf{z}_{1:L} \sim q_{\theta}(\mathbf{z}_{1:L} \mid \mathbf{x})} \left[\log \frac{p_{\phi}(\mathbf{x}, \mathbf{z}_{1:L})}{q_{\theta}(\mathbf{z}_{1:L} \mid \mathbf{x})} \right]}_{\mathcal{L}_{\text{H-ELBO}}(\theta, \phi; \mathbf{x})}.$$

An HVAE applies the same variational principle to an entire latent hierarchy rather than to one latent variable.

Why Is a Deep HVAE Difficult to Train?

The hierarchy increases expressiveness, but it also requires learning multiple stochastic transformations in both directions:

$$\underbrace{\mathbf{x} \rightarrow \mathbf{z}_1 \rightarrow \cdots \rightarrow \mathbf{z}_L}_{\text{learned inference}} \quad \underbrace{\mathbf{z}_L \rightarrow \mathbf{z}_{L-1} \rightarrow \cdots \rightarrow \mathbf{z}_1 \rightarrow \mathbf{x}}_{\text{learned generation}}.$$

This creates a major challenge:

- **Difficult joint optimization.** The bottom-up inference model and the top-down generative model must learn compatible distributions at every latent level.
- **Weak learning signals.** Reconstruction is computed from $\mathbf{z}_1$, so higher-level variables influence it only indirectly through several stochastic transitions.

Can we retain a deep stochastic hierarchy while avoiding the need to learn the entire inference process?

Lecture 1: Main Takeaways

- 1 Generative modeling aims to learn a distribution from which new observations can be sampled.
- 2 VAEs introduce a probabilistic latent space and maximize an ELBO on the intractable marginal log-likelihood.
- 3 HVAEs distribute stochastic information across multiple latent levels.

How can we retain a deep stochastic hierarchy while avoiding the need to learn the entire inference process?

4 Appendix

Appendix: Convex and Concave Functions

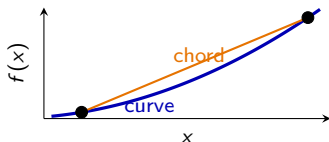
For x_1, x_2 and $\lambda \in [0, 1]$,

$$\bar{x} = \lambda x_1 + (1 - \lambda)x_2$$

is a weighted average of x_1 and x_2 .

Convex function

$$\begin{aligned} & f(\lambda x_1 + (1 - \lambda)x_2) \\ & \leq \lambda f(x_1) + (1 - \lambda)f(x_2) \end{aligned}$$

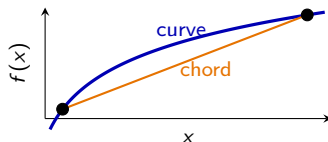


The chord lies **above** the curve.

x^2
convex

Concave function

$$\begin{aligned} & f(\lambda x_1 + (1 - \lambda)x_2) \\ & \geq \lambda f(x_1) + (1 - \lambda)f(x_2) \end{aligned}$$



The chord lies **below** the curve.

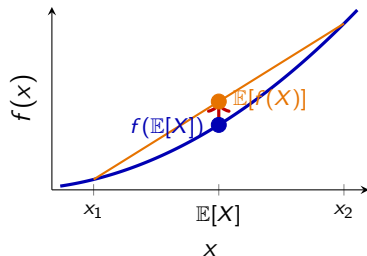
$\log x$
concave

Appendix: Jensen's Inequality

For a random variable X , Jensen's inequality compares the **function of the average** with the **average function value**.

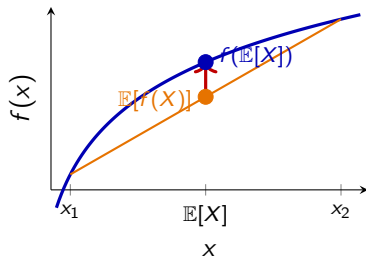
Convex f

$$f(\mathbb{E}[X]) \leq \mathbb{E}[f(X)]$$



Concave f

$$f(\mathbb{E}[X]) \geq \mathbb{E}[f(X)]$$



In our derivations:

$$\underbrace{\mathbb{E}[-\log X] \geq -\log \mathbb{E}[X]}_{\text{KL proof: } -\log \text{ is convex}}$$

$$\underbrace{\log \mathbb{E}[X] \geq \mathbb{E}[\log X]}_{\text{ELBO: log is concave}}$$

[◀ Back to ELBO derivation](#)

Appendix: Forward KL Properties: A Short Proof

$$\begin{aligned}\mathcal{D}_{\text{KL}}(p_{\text{data}} \| p_{\phi}) &= \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} \left[\log \frac{p_{\text{data}}(\mathbf{x})}{p_{\phi}(\mathbf{x})} \right] \\ &= \mathbb{E}_{p_{\text{data}}} \left[-\log \frac{p_{\phi}(\mathbf{x})}{p_{\text{data}}(\mathbf{x})} \right] \stackrel{\substack{\text{Jensen's inequality} \\ -\log(\cdot) \text{ is convex}}}{\geq} -\log \mathbb{E}_{p_{\text{data}}} \left[\frac{p_{\phi}(\mathbf{x})}{p_{\text{data}}(\mathbf{x})} \right] \\ &= -\log \int p_{\phi}(\mathbf{x}) \, d\mathbf{x} = -\log 1 = 0.\end{aligned}$$

① $\mathcal{D}_{\text{KL}}(p_{\text{data}} \| p_{\phi}) \geq 0$.

② Equality in Jensen's inequality holds only when

$$\frac{p_{\phi}(\mathbf{x})}{p_{\text{data}}(\mathbf{x})} = 1 \quad \text{and hence } p_{\phi} = p_{\text{data}}$$

③ If $p_{\phi}(\mathbf{x}) \ll p_{\text{data}}(\mathbf{x})$, then

$\log \frac{p_{\text{data}}(\mathbf{x})}{p_{\phi}(\mathbf{x})}$ is large, producing a large penalty.

Appendix: When Is the ELBO Tight?

Using Bayes' rule: $p_{\phi}(\mathbf{z} \mid \mathbf{x}) = \frac{p_{\phi}(\mathbf{x}, \mathbf{z})}{p_{\phi}(\mathbf{x})}$ the divergence between the approximate and exact posteriors becomes

$$\begin{aligned}\mathcal{D}_{\text{KL}}(q_{\theta}(\mathbf{z} \mid \mathbf{x}) \parallel p_{\phi}(\mathbf{z} \mid \mathbf{x})) &= \mathbb{E}_{q_{\theta}(\mathbf{z} \mid \mathbf{x})} \left[\log \frac{q_{\theta}(\mathbf{z} \mid \mathbf{x}) p_{\phi}(\mathbf{x})}{p_{\phi}(\mathbf{x}, \mathbf{z})} \right] \\ &= \log p_{\phi}(\mathbf{x}) - \mathcal{L}_{\text{ELBO}}(\theta, \phi; \mathbf{x}).\end{aligned}$$

Therefore, the marginal log-likelihood decomposes exactly as

$$\log p_{\phi}(\mathbf{x}) = \mathcal{L}_{\text{ELBO}}(\theta, \phi; \mathbf{x}) + \underbrace{\mathcal{D}_{\text{KL}}(q_{\theta}(\mathbf{z} \mid \mathbf{x}) \parallel p_{\phi}(\mathbf{z} \mid \mathbf{x}))}_{\text{approximation gap}}$$

Since KL divergence is non-negative,

$$\mathcal{L}_{\text{ELBO}}(\theta, \phi; \mathbf{x}) \leq \log p_{\phi}(\mathbf{x})$$

The ELBO is tight exactly when $q_{\theta}(\mathbf{z} \mid \mathbf{x}) = p_{\phi}(\mathbf{z} \mid \mathbf{x})$; then the KL gap is zero.

[◀ Back to ELBO derivation](#)